

CSE 225L Lab 4

Template Class & Operator Overloading
Implementation, Dry Runs, Debugging, and Complete
Solutions

Language: C++

Topic: Templates

Topic: Complex Operators

IDE: Code::Blocks / g++

Built from the uploaded Lab 4 sheet. The guide expands the short lab into a practical coding handbook: what to write, why it works, how memory behaves, how to test it, and how to answer viva questions.

1. Lab Map

Lab 4 is the bridge from fixed-type classes to reusable C++ abstractions. The original `dynArr` stores only `int`. This lab turns it into `dynArr<T>`, so the same class can store integers, doubles, characters, or even user-defined objects. The second half builds a `Complex` class and overloads `*` and `!=`, making object arithmetic feel natural.

Lab Area	What You Build	C++ Concepts	Skill Gained
Template dynamic array	A reusable <code>dynArr<T></code> class	<code>template<class T></code> , constructors, destructor, heap array	Write one class that works for multiple data types.
Operator overloading	A <code>Complex</code> class with <code>operator*</code> and <code>operator!=</code>	Classes, private fields, member operators, return objects, boolean comparisons	Make custom objects behave like built-in values.
Project structure	Separate source sets for array template and complex class	Header files, source files, driver files, include guards	Submit clean multi-file C++ projects.

Instructor-facing outcome

By the end, you should be able to explain why templates usually keep implementation visible in headers, how a generic heap array is allocated, how complex multiplication is calculated, and how C++ translates `a * b` into a member function call.

2. Page Extraction Map

This table is the lab sheet x-ray. Every visible page is mapped before solution construction.

LAB 4 CSE225L  Template Class & Operator Overloading In this lab, we will: - Modify the <code>dynArr</code> class to work as a template class so the array elements can be of any type defined by the user. - Update the header (.h) and source (.cpp) files to support dynamic memory allocation for any data type. - Build a complex number class. - Overload the <code>*</code> (multiplication) and <code>!=</code> (not equal) operators for the <code>Complex</code> class to support complex number arithmetic and comparison. - Implement necessary methods to handle these operations.	TEMPLATE CLASS Task 1: Modify the given header (.h) and source (.cpp) files to implement a template class that allows dynamic memory allocation for array elements of any user-defined type. <pre>dynarr.h #ifndef DYNARR_H #define DYNARR_H class dynArr { private: int *data; int size; public: dynArr(); dynArr(int s); ~dynArr(); void setValue(int index, int value); int getValue(int index); }; #endif // DYNARR_H dynarr.cpp #include "dynarr.h" #include <iostream> using namespace std; dynArr::dynArr() { data = NULL; size = 0; } dynArr::dynArr(int s) { data = new int[s]; size = s; } dynArr::~dynArr() { delete[] data; } int dynArr::getValue(int index) { return data[index]; } void dynArr::setValue(int index, int value) { data[index] = value; }</pre>	OPERATOR OVERLOADING Task 2: Modify the given Complex number class to overload the <code>*</code> (multiplication) and <code>!=</code> (not equal) operators. In the driver file (main.cpp), include the Complex class, create two objects, and demonstrate the usage of the <code>*</code> and <code>!=</code> operators between them. <pre>complex.h #ifndef COMPLEX_H #define COMPLEX_H class Complex { private: double real, imaginary; public: Complex(); Complex(double r, double i); Complex operator+(Complex); void Print(); }; #endif // COMPLEX_H complex.cpp #include "complex.h" #include <iostream> using namespace std; Complex::Complex() { real = 0; imaginary = 0; } Complex::Complex(double r, double i) { real = r; imaginary = i; } Complex Complex::operator+(Complex a) { Complex t; t.real = real + a.real; t.imaginary = imaginary + a.imaginary; return t; } void Complex::Print() { cout << "Real: " << real << ", Imaginary: " << imaginary << endl; }</pre>
Source Page 1	Source Page 2	Source Page 3

Page	Visible Topic	Extracted Requirements	Given Code / Notes	Status
1	Lab 4 overview: Template Class & Operator Overloading	Modify <code>dynArr</code> into a template class, update header/source files, build <code>Complex</code> , overload <code>*</code> and <code>!=</code> .	No code on this page. It states learning outcomes and class/operator targets.	Explained and expanded.
2	Template Class - Task 1	Convert the given int-only <code>dynArr</code> into a template class that dynamically allocates an array of any user-defined type.	Starter files: <code>dynarr.h</code> with <code>int* data</code> , <code>setValue(int,int)</code> , <code>int getValue(int)</code> ; <code>dynarr.cpp</code> with constructors/destructor/accessors.	Corrected, implemented, traced, tested.
3	Operator Overloading - Task 2	Modify the provided <code>Complex</code> class to overload multiplication and not-equal operators. Main creates two objects and demonstrates both.	Starter files: <code>complex.h</code> with constructors, <code>operator+</code> , <code>Print()</code> ; <code>complex.cpp</code> with addition and printing.	Corrected, implemented, traced, tested.

Important correction from the lab sheet

The sheet asks for template code in header and source files. In C++, template method definitions must be visible to the compiler at the point where `dynArr<int>` or `dynArr<double>` is created. The safest beginner submission keeps the template implementation in `dynarr.h`. This guide explains why, while still teaching the original header/source design idea.

3. Implementation Toolkit

What This Lab Is Really About

This lab is not only about adding the word `template`. It is about building one reusable class blueprint and letting the compiler manufacture type-specific versions only when needed. It is also not only about writing `operator*`; it is about teaching C++ what multiplication means for a custom object.

Concepts Before Code

Task	Data Needed	Main C++ Tool	Object / Memory Created	What Can Go Wrong
Template <code>dynArr<T></code>	Array size and values of type <code>T</code>	Template class, constructor, destructor, pointer	Object on stack contains <code>T*</code> ; heap stores <code>T[size]</code> .	Putting template definitions only in <code>.cpp</code> can cause linker errors.
Complex multiplication	Two complex numbers: real and imaginary parts	Member operator overload	Each object stores two <code>double</code> values; product returns a new object.	Wrong formula: using <code>ac + bd</code> instead of <code>ac - bd</code> .
Not-equal comparison	Two complex numbers	Boolean operator overload	No heap memory required.	Using <code>&&</code> instead of <code> </code> makes unequal numbers look equal.

How to Recognize the Coding Pattern

Task Pattern	C++ Tool Needed	First Move	Memory Concern	Common Trap
Generic class	<code>template<class T></code>	Replace fixed type names with <code>T</code> .	If <code>T*</code> points to heap memory, destructor must use <code>delete[]</code> .	Leaving <code>int</code> in <code>setValue</code> or <code>getValue</code> .
Dynamic 1D array inside class	<code>new[]</code> , <code>delete[]</code>	Constructor allocates; destructor frees.	Every <code>new T[s]</code> needs <code>delete[]</code> <code>data</code> .	Using <code>delete</code> instead of <code>delete[]</code> .
Header/source split with templates	Header-visible definitions	Keep template implementation in header or include implementation from header.	No extra heap concern, but build system concern.	Putting definitions only in <code>dynarr.cpp</code> and compiling separately.
Operator multiplication	Member <code>operator*</code>	Decide formula and return type.	Usually none, unless operator allocates memory.	Returning <code>void</code> instead of a new object.
		Return <code>bool</code> .	None.	

Task Pattern	C++ Tool Needed	First Move	Memory Concern	Common Trap
Operator comparison	Member <code>operator!=</code>			Forgetting that unequal means either field differs.

Lab Decision Tree

Decision Tree: What should I write first?

1. Is the task asking for one class to support many data types?
 -> Start with `template<class T>` above the class.
 -> Replace `int*` with `T*`, `int` value with `T` value, `int` return type with `T`.
2. Does the class own heap memory?
 -> Constructor allocates using `new T[size]`.
 -> Destructor releases using `delete[] data`.
 -> Never forget size, because loops need it.
3. Is the task asking for object arithmetic?
 -> Write a member operator function.
 -> Decide return type: Complex for multiplication, `bool` for comparison.
 -> Test using normal syntax: `c1 * c2` and `c1 != c2`.
4. Is it a multi-file task?
 -> For normal classes: declaration in `.h`, definitions in `.cpp`.
 -> For templates: definitions must be visible, so header-only is the safest route.
 -> Compile all ordinary `.cpp` files in `Code::Blocks`.

Required C++ Prerequisite Bridge

Templates

A template is a code stencil. `dynArr<int>` creates an `int` version; `dynArr<double>` creates a `double` version. The compiler does the copying and type replacement.

Operator overloading

Operator overloading lets you define how operators work for your own class.

`c1 * c2` becomes `c1.operator*(c2)`.

4. Concept and Implementation Foundation

Template Class Foundation

Transformation: int-only dynArr becomes generic dynArr<T>

Before

```
int* data;  
void setValue(int, int);  
int getValue(int);
```

template<class T>



After

```
T* data;  
void setValue(int, T);  
T getValue(int) const;  
dynArr<int>, dynArr<double>, dynArr<Comple
```

The original `dynArr` from the lab sheet is locked to `int`. To make it generic, identify every place where the stored element type appears. The pointer becomes `T*`, the input value becomes `T value`, and `getValue` returns `T`.

Memory model: object on stack, template array on heap

Stack frame: main()

```
dynArr<int> a  
data = 0xA100  
size = 5
```

pointer address

Heap: new T[size]

10	20	30	40	50
----	----	----	----	----

delete[] data in destructor

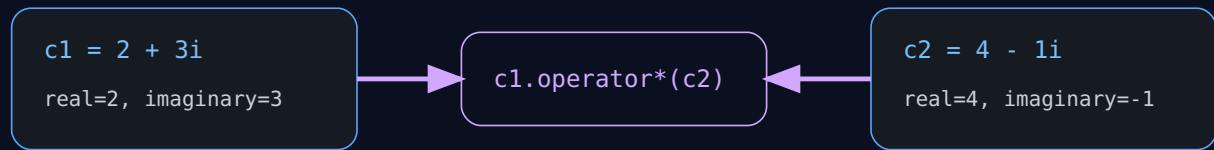
Template Implementation Visibility

Why template code usually lives in the header

Normal class methods can live in a `.cpp` file because the compiler creates one fixed function body. Template classes are different: the compiler must generate a version for each type, such as `dynArr<int>` and `dynArr<double>`. If the method bodies are hidden in a separate `.cpp`, the compiler may know the function exists but cannot generate it. Result: linker errors lurking in the hallway wearing a trench coat.

Operator Overloading Foundation

Operator calls and complex multiplication



$$(a+bi)(c+di) = (ac - bd) + (ad + bc)i$$
$$(2+3i)(4-1i) = (8 - -3) + (-2 + 12)i = 11 + 10i$$

For two complex numbers $a + bi$ and $c + di$, multiplication is $(ac - bd) + (ad + bc)i$. In the class, `real` and `imaginary` belong to the left object, and the parameter object contains the right object.

Expression	Compiler Interpretation	Return Type	Meaning
<code>c1 * c2</code>	<code>c1.operator*(c2)</code>	<code>Complex</code>	Return the product as a new complex number.
<code>c1 != c2</code>	<code>c1.operator!=(c2)</code>	<code>bool</code>	Return true if real parts or imaginary parts differ.
<code>product.Print()</code>	normal member function call	<code>void</code>	Display stored real and imaginary values.

5. Algorithm Workflows

Workflow A: Convert int dynArr into template dynArr<T>

Algorithm: Template Conversion

Input: The original int-only dynArr class.

Output: A generic dynArr<T> class.

1. Add `template<class T>` before the class declaration.
2. Change `int* data` to `T* data`.
3. Keep `int size`, because `size` is a count, not an element.
4. Change `setValue(int index, int value)` to `setValue(int index, T value)`.
5. Change `int getValue(int index)` to `T getValue(int index) const`.
6. In constructor, allocate `data = new T[s]`.
7. In destructor, deallocate using `delete[] data`.
8. Test with at least two types, such as `int` and `double`.

Workflow B: Use a Template Dynamic Array

Algorithm: Store and Print Values

Input: size and values.

Output: printed array values.

1. Create `dynArr<int> numbers(5)`.
2. Loop `index` from 0 to `size - 1`.
3. Read one value from the user.
4. Call `numbers.setValue(index, value)`.
5. Loop again from 0 to `size - 1`.
6. Print `numbers.getValue(index)`.
7. When `main` ends, destructor runs automatically and frees heap memory.

Workflow C: Complex Multiplication Operator

Algorithm: `operator*` for Complex

Input: left object = `this`, right object = `a`.

Output: a new Complex object named `t`.

1. Create a temporary Complex object `t`.
2. `t.real = real * a.real - imaginary * a.imaginary`.
3. `t.imaginary = real * a.imaginary + imaginary * a.real`.
4. Return `t`.
5. In `main`, write `Complex product = c1 * c2`.

Workflow D: Complex Not-Equal Operator

Algorithm: `operator!=` for Complex

Input: left object = `this`, right object = `a`.

Output: `true` or `false`.

1. Compare real parts.
2. Compare imaginary parts.
3. If either pair is different, return `true`.
4. Otherwise return `false`.
5. In `main`, write `if (c1 != c2)`.

SECTION 6A

Complete Task Solution 1

Template Class: Modify `dynArr` to work with any type.

6A. Task 1 - Template dynArr<T>

Task Statement

Modify the given header and source files to implement a template class that allows dynamic memory allocation for array elements of any user-defined type.

Starter Code Analysis

The lab provides an int-only class. The design is good for Lab 3, but too narrow for Lab 4.

Given starter file: dynarr.h

```
#ifndef DYNARR_H
#define DYNARR_H

class dynArr
{
private:
    int *data;
    int size;

public:
    dynArr();
    dynArr(int s);
    ~dynArr();
    void setValue(int index, int value);
    int getValue(int index);
};

#endif // DYNARR_H
```

Given starter file: dynarr.cpp

```
#include "dynarr.h"
#include <iostream>
using namespace std;

dynArr::dynArr()
{
    data = NULL;
    size = 0;
}

dynArr::dynArr(int s)
{
    data = new int[s];
    size = s;
}

dynArr::~dynArr()
{
    delete[] data;
}

int dynArr::getValue(int index)
{
    return data[index];
}

void dynArr::setValue(int index, int value)
{
}
```

```
data[index] = value;
}
```

Original Part	Problem for Template Lab	Correction
<code>int *data</code>	Only stores integers.	Change to <code>T* data</code> .
<code>void setValue(int, int)</code>	Second parameter cannot accept double, char, Complex, etc.	Change to <code>void setValue(int, T)</code> .
<code>int getValue(int)</code>	Always returns int even if array stores another type.	Change to <code>T getValue(int) const</code> .
Method bodies in <code>.cpp</code>	Templates may fail to link if bodies are hidden in a separately compiled source file.	Use header-visible template definitions.

What You Need to Know First

Construction plan

Keep `size` as `int` because it counts elements. Replace the stored element type with `T`. Allocate with `new T[s]`. Delete with `delete[] data`. Add a small `getSize()` helper so `main()` can loop cleanly without hardcoding the array length.

Pseudocode

Algorithm: Generic dynArr Class

```
Class dynArr<T>
Private:
    T* data
    int size

Constructor with no argument:
    data = NULL
    size = 0

Constructor with size s:
    if s <= 0, create an empty array
    otherwise allocate data = new T[s]
    store size = s

setValue(index, value):
    if index is valid, write value into data[index]

getValue(index):
    if index is valid, return data[index]
    otherwise return default T()

Destructor:
    delete[] data
```

Required File Structure

Folder	File	Purpose
template_array	<code>dynarr.h</code>	

Folder	File	Purpose
		Template class declaration and definitions. This is the safest C++ template pattern.
template_array	main.cpp	Creates <code>dynArr<int></code> and <code>dynArr<double></code> , reads/prints values, tests generic behavior.
template_array	dynarr.cpp	Not required in the clean template solution. If an instructor insists on a separate implementation file, it must be included from the header or converted to a <code>.hpp</code> pattern.

Full C++ Code

● ● ● template_array/dynarr.h

```

#ifndef DYNARR_H
#define DYNARR_H

#include <iostream>
using namespace std;

template <class T>
class dynArr {
private:
    T* data;
    int size;

public:
    dynArr();
    dynArr(int s);
    ~dynArr();

    void setValue(int index, T value);
    T getValue(int index) const;
    int getSize() const;
};

template <class T>
dynArr<T>::dynArr() {
    data = NULL;
    size = 0;
}

template <class T>
dynArr<T>::dynArr(int s) {
    if (s <= 0) {
        data = NULL;
        size = 0;
    } else {
        data = new T[s];
        size = s;
    }
}

template <class T>
dynArr<T>::~dynArr() {
    delete[] data;
}

template <class T>
void dynArr<T>::setValue(int index, T value) {
    if (index >= 0 && index < size) {
        data[index] = value;
    }
}

template <class T>
T dynArr<T>::getValue(int index) const {
    if (index >= 0 && index < size) {

```

```

        return data[index];
    }
    return T();
}

template <class T>
int dynArr<T>::getSize() const {
    return size;
}

#endif // DYNARR_H

```

● ● ● template_array/main.cpp

```

#include <iostream>
#include "dynarr.h"
using namespace std;

int main() {
    dynArr<int> numbers(5);

    cout << "Enter 5 integer values: ";
    for (int i = 0; i < numbers.getSize(); i++) {
        int value;
        cin >> value;
        numbers.setValue(i, value);
    }

    cout << "Integer array: ";
    for (int i = 0; i < numbers.getSize(); i++) {
        cout << numbers.getValue(i) << " ";
    }
    cout << endl;

    dynArr<double> marks(3);
    marks.setValue(0, 87.5);
    marks.setValue(1, 91.25);
    marks.setValue(2, 78.0);

    cout << "Double array: ";
    for (int i = 0; i < marks.getSize(); i++) {
        cout << marks.getValue(i) << " ";
    }
    cout << endl;

    return 0;
}

```

Compiled Sample Run

```

Enter 5 integer values: Integer array: 10 20 30 40 50
Double array: 87.5 91.25 78

```

Code Explanation

Code Part	Purpose	What to Watch
<code>template <class T></code>	Makes the class generic.	Write it before every out-of-class template method definition if methods are outside the class body.
<code>T* data</code>	Pointer to a heap array of type <code>T</code> .	Still needs <code>delete[]</code> because it points to an array.
<code>new T[s]</code>		If <code>s</code> is invalid, avoid allocating.

Code Part	Purpose	What to Watch
	Allocates an array using whatever type the object chooses.	
<code>T getValue(int index)</code> <code>const</code>	Returns an element of the same stored type.	Returning <code>T()</code> gives a safe default for bad index.
<code>dynArr<double></code> <code>marks(3)</code>	Proves the class is not int-only anymore.	If this line compiles, the template worked.

Dry Run

Step	Object State	Action	Output
1	<code>numbers.data -> heap[5]</code> , <code>size=5</code>	Create <code>dynArr<int> numbers(5)</code> .	None.
2	Index 0 to 4	Read <code>10 20 30 40 50</code> , call <code>setValue</code> .	Prompt appears.
3	Heap contains <code>[10,20,30,40,50]</code>	Loop and call <code>getValue</code> .	Integer array: <code>10 20 30 40 50</code>
4	<code>marks.data -> heap[3]</code> , <code>size=3</code>	Store double values.	Double array: <code>87.5 91.25 78</code>
5	Objects leaving <code>main()</code>	Destructor runs automatically.	Heap memory freed.

Common bugs

- `undefined reference to dynArr<int>::setValue`: template definitions are hidden in a separate `.cpp`.
- Output is garbage: you read from an index that was never assigned.
- Crash: invalid index or object with size 0.
- Memory leak: constructor uses `new[]`, but destructor missing `delete[]`.

Edge Cases and Complexity

Case	Expected Behavior
<code>s <= 0</code>	Object becomes empty: <code>data = NULL</code> , <code>size = 0</code> .
Bad index	This beginner-safe version ignores bad writes and returns <code>T()</code> for bad reads.
Different type	<code>dynArr<double></code> and <code>dynArr<char></code> compile without rewriting the class.

Operation	Time Complexity	Space Complexity
Constructor allocation	$O(n)$	$O(n)$ heap memory

Operation	Time Complexity	Space Complexity
<code>setValue</code>	O(1)	O(1)
<code>getValue</code>	O(1)	O(1)
Destructor	O(1) for releasing array handle; runtime frees array storage	O(1) extra

Viva Questions

Question	Short Answer
Why use a template class?	To write one class that works with multiple data types.
Why does template implementation stay in the header?	Because the compiler needs method bodies to generate type-specific versions.
What does <code>T*</code> mean?	A pointer to an array or object of the template type <code>T</code> .
Why use <code>delete[]</code> ?	Because the memory was allocated using <code>new[]</code> .
What is <code>T()</code> ?	A default value for the type, such as 0 for int or 0.0 for double.

SECTION 6B

Complete Task Solution 2

Complex Class: Overload `*` and `!=`.

6B. Task 2 - Complex Operator Overloading

Task Statement

Modify the given `Complex` number class to overload the `*` multiplication operator and the `!=` not-equal operator. In `main.cpp`, create two objects and demonstrate both operations.

Starter Code Analysis

Given starter file: `complex.h`

```
#ifndef COMPLEX_H
#define COMPLEX_H

class Complex
{
private:
    double real, imaginary;

public:
    Complex();
    Complex(double, double);
    Complex operator+(Complex);
    void Print();
};

#endif // COMPLEX_H
```

Given starter file: `complex.cpp`

```
#include "complex.h"
#include <iostream>
using namespace std;

Complex::Complex()
{
    real = 0;
    imaginary = 0;
}

Complex::Complex(double r, double i)
{
    real = r;
    imaginary = i;
}

Complex Complex::operator+(Complex a)
{
    Complex t;
    t.real = real + a.real;
    t.imaginary = imaginary + a.imaginary;
    return t;
}

void Complex::Print()
{
    cout << "Real: " << real << ", Imaginary: " << imaginary << endl;
}
```

Original Part	What It Does	What Must Be Added
<code>Complex()</code>	Creates <code>0 + 0i</code> .	Keep it.
<code>Complex(double,double)</code>	Creates a complex number from real and imaginary values.	Keep it.
<code>operator+</code>	Adds two complex numbers.	Use it as a model, but implement multiplication formula separately.
<code>Print()</code>	Displays object state.	Use it to show product.
Missing <code>operator*</code>	No multiplication support.	Return a new <code>Complex</code> .
Missing <code>operator!=</code>	No custom comparison support.	Return <code>bool</code> .

What You Need to Know First

Construction plan

`operator*` should not print directly. It should calculate and return a new `Complex` object, so the caller can store it. `operator!=` should return `true` if either real parts are different or imaginary parts are different.

Pseudocode

Algorithm: `Complex operator* and operator!=`

```
operator*(Complex a):
    Create temporary Complex t
    t.real = real * a.real - imaginary * a.imaginary
    t.imaginary = real * a.imaginary + imaginary * a.real
    return t

operator!=(Complex a):
    if real != a.real OR imaginary != a.imaginary:
        return true
    else:
        return false
```

Required File Structure

Folder	File	Purpose
complex_operator	<code>complex.h</code>	Class declaration and operator prototypes.
complex_operator	<code>complex.cpp</code>	Constructor and operator definitions.
complex_operator	<code>main.cpp</code>	Creates objects and demonstrates <code>*</code> and <code>!=</code> .

Full C++ Code

complex_operator/complex.h

```

#ifndef COMPLEX_H
#define COMPLEX_H

class Complex {
private:
    double real, imaginary;

public:
    Complex();
    Complex(double r, double i);

    Complex operator+(Complex a);
    Complex operator*(Complex a);
    bool operator!=(Complex a);

    void Print();
};

#endif // COMPLEX_H

```

● ● ● complex_operator/complex.cpp

```

#include "complex.h"
#include <iostream>
using namespace std;

Complex::Complex() {
    real = 0;
    imaginary = 0;
}

Complex::Complex(double r, double i) {
    real = r;
    imaginary = i;
}

Complex Complex::operator+(Complex a) {
    Complex t;
    t.real = real + a.real;
    t.imaginary = imaginary + a.imaginary;
    return t;
}

Complex Complex::operator*(Complex a) {
    Complex t;
    t.real = (real * a.real) - (imaginary * a.imaginary);
    t.imaginary = (real * a.imaginary) + (imaginary * a.real);
    return t;
}

bool Complex::operator!=(Complex a) {
    return real != a.real || imaginary != a.imaginary;
}

void Complex::Print() {
    cout << "Real: " << real << ", Imaginary: " << imaginary << endl;
}

```

● ● ● complex_operator/main.cpp

```

#include <iostream>
#include "complex.h"
using namespace std;

int main() {
    Complex c1(2, 3);
    Complex c2(4, -1);

    cout << "First complex number: ";
    c1.Print();
}

```

```

cout << "Second complex number: ";
c2.Print();

Complex product = c1 * c2;
cout << "Product: ";
product.Print();

if (c1 != c2) {
    cout << "The two complex numbers are not equal." << endl;
} else {
    cout << "The two complex numbers are equal." << endl;
}

return 0;
}

```

Compiled Sample Run

```

First complex number: Real: 2, Imaginary: 3
Second complex number: Real: 4, Imaginary: -1
Product: Real: 11, Imaginary: 10
The two complex numbers are not equal.

```

Code Explanation

Code Part	Purpose	What to Watch
<code>Complex operator*(Complex a)</code>	Defines multiplication between two Complex objects.	Return <code>Complex</code> , not <code>void</code> .
<code>t.real = (real * a.real) - ...</code>	Computes real part using <code>ac - bd</code> .	The minus sign is essential because $i^2 = -1$.
<code>t.imaginary = ...</code>	Computes imaginary part using <code>ad + bc</code> .	Do not accidentally subtract the cross terms.
<code>bool operator!=(Complex a)</code>	Defines inequality comparison.	Use OR: unequal if either field differs.
<code>Complex product = c1 * c2</code>	Tests overloaded multiplication.	This is cleaner than calling <code>c1.operator*(c2)</code> directly.

Dry Run

Step	Expression / State	Calculation	Result
1	<code>c1 = 2 + 3i</code> , <code>c2 = 4 - 1i</code>	Objects are initialized.	Ready for operation.
2	<code>c1 * c2</code>	$\text{real} = 2*4 - 3*(-1) = 8 + 3$	11
3	<code>c1 * c2</code>	$\text{imaginary} = 2*(-1) + 3*4 = -2 + 12$	10
4	<code>product</code>	Product stores <code>11 + 10i</code> .	Printed as Real: 11, Imaginary: 10.
5	<code>c1 != c2</code>	<code>2 != 4</code> is true, so inequality is true.	Prints not equal.

Common bugs

- Formula bug: complex multiplication is not component-wise multiplication.
- Return bug: `operator*` must return the temporary object.
- Comparison bug: use `||`, not `&&`.
- Linker bug: add `complex.cpp` to the Code::Blocks project, otherwise `undefined reference` errors appear.

Edge Cases and Complexity

Case	Expected Behavior
Both objects equal	<code>operator!=</code> returns false.
Only real differs	Returns true.
Only imaginary differs	Returns true.
Multiplication by zero complex number	Product becomes <code>0 + 0i</code> .
Decimal values	Works because fields are <code>double</code> .

Operation	Time Complexity	Space Complexity
Constructor	O(1)	O(1)
<code>operator*</code>	O(1)	O(1)
<code>operator!=</code>	O(1)	O(1)
<code>Print()</code>	O(1)	O(1)

Viva Questions

Question	Short Answer
What does operator overloading mean?	Giving custom meaning to operators for user-defined classes.
What does <code>c1 * c2</code> call internally?	It calls <code>c1.operator*(c2)</code> .
What is the complex multiplication formula?	$(a+bi)(c+di) = (ac-bd) + (ad+bc)i$.
Why does <code>operator!=</code> return bool?	Because a comparison should answer true or false.
Why are <code>real</code> and <code>imaginary</code> private?	To protect internal data and control access through class methods.

7. Debugging and Common Errors

Symptom	Likely Cause	Fix
undefined reference to <code>dynArr<int>...</code>	Template definitions were placed only in <code>dynarr.cpp</code> .	Move definitions into <code>dynarr.h</code> or include a template implementation file from the header.
expected initializer before <code><</code> token	Forgot <code>template<class T></code> before an out-of-class method definition.	Write <code>template <class T></code> before every template method body.
Garbage output from array	Reading uninitialized array cells or invalid index.	Only print indices from 0 to <code>size-1</code> ; initialize values first.
Program crashes after running	Bad array access or delete mismatch.	Check bounds and use <code>delete[]</code> for arrays.
Complex product wrong	Formula implemented incorrectly.	Use <code>real = ac - bd</code> and <code>imaginary = ad + bc</code> .
Complex files compile separately but main fails	<code>complex.cpp</code> not added to project.	In Code::Blocks, make sure <code>complex.cpp</code> is under Sources.

Code::Blocks build note

For the Complex task, compile `main.cpp` and `complex.cpp` together. For the template `dynArr` task, the clean solution only needs `main.cpp` and `dynarr.h`, because template method definitions are inside the header.

8. Testing Checklist

✓	Test	Why It Matters
☑	Create <code>dynArr<int></code> with size 5.	Confirms basic template array allocation.
☑	Create <code>dynArr<double></code> .	Proves the class is generic, not secretly int-only.
☑	Use index 0 and last index.	Checks array boundaries.
☑	Try invalid size such as 0.	Checks safe constructor behavior.
☑	Multiply <code>2+3i</code> and <code>4-1i</code> .	Known expected answer is <code>11+10i</code> .
☑	Compare unequal complex numbers.	Checks <code>operator!=</code> true branch.
☑	Compare identical complex numbers.	Checks <code>operator!=</code> false branch.
☑	Build from clean project.	Catches missing source files and template linkage issues.

9. Complexity Summary

Component	Operation	Time	Extra Space	Reason
Template dynArr	Constructor with size n	O(n) allocation request	O(n)	Allocates n cells on heap.
Template dynArr	setValue(index,value)	O(1)	O(1)	Direct index access.
Template dynArr	getValue(index)	O(1)	O(1)	Direct index access.
Template dynArr	Destructor	O(1) handle operation	O(1)	Releases owned heap array.
Complex	operator*	O(1)	O(1)	Fixed number of arithmetic operations.
Complex	operator!=	O(1)	O(1)	Two comparisons.
Complex	Print	O(1)	O(1)	Prints two fields.

10. Quick Revision Sheet

Syntax Patterns

- `template <class T>`
- `class dynArr { T* data; int size; };`
- `dynArr<int> a(5);`
- `Complex operator*(Complex a);`
- `bool operator!=(Complex a);`

Pointer Safety Rules

- `new T[s]` pairs with `delete[] data`.
- Never read outside `0` to `size-1`.
- Set empty objects to `NULL` and size `0`.
- Destructor should not print random messages unless instructor wants it.

Final Submission Checklist

- Template task compiles with at least two data types.
- Complex task demonstrates both `*` and `!=`.
- All headers have include guards.
- All required `.cpp` files are added to the project.
- Output is clean and readable.

11. Lab Coverage Audit

Page / Slide Range	Task or Topic Found	Where It Was Covered	Status	Notes
Page 1	Lab title and goals: Template Class & Operator Overloading, template <code>dynArr</code> , dynamic memory for any type, Complex class, overload <code>*</code> and <code>!=</code> .	Lab Map, Concept Foundation, Complete Solutions	Explained	All learning goals converted into guide modules.
Page 2	Task 1: Convert given <code>dynArr</code> header/source starter code into a template class.	Section 6A	Implemented, Traced, Tested, Corrected	Starter code preserved in analysis; final version uses header-visible template implementation.
Page 3	Task 2: Modify <code>Complex</code> class to overload multiplication and not equal operators; demonstrate in driver file.	Section 6B	Implemented, Traced, Tested	Starter addition operator retained; multiplication and inequality added.
All pages	No unreadable diagrams or tables found.	Extraction Map and Coverage Audit	Complete	Source pages are included as thumbnails in Section 2.

Validation note

The included C++ files were compiled with `g++ -std=c++11`. The sample outputs in the terminal panels come from actual compiled runs.